PROJECTREPORTFOR

FoundationsofArtificialIntelligence(CS23533)

RIPENSENSE AI

A PROJECT REPORT

Submittedby

2116240701096– Darun Vignesh R K

2116240701096– Arvind Harish Nataraj



November,2026

BONAFIDECERTIFICATE

Certified that this project report "**RIPENSENSE AI**" is the bonafide work of "Darun Vignesh R K (240701096) & Arvind Harish Nataraj (240701052) who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLEOF CONTENTS

CHAPTERNO.	TITLE	PAGE NO.
	ABSTRACT	1
1	INTRODUCTION	3
2	LITERATURE REVIEW	7
3	PROPOSED SYSTEM	10
	3.1 ARCHITECTURE DIAGRAM	11
4	MODULES DESCRIPTION	21
	4.1 MODULE 1 – IMAGE INPUT AND VALIDATION	21
	4.2 MODULE 2 – IMAGE PROCESSING	22
5	IMPLEMENTATION AND RESULTS	26
	5.1 EXPERIMENTAL SETUP	26
	5.2 RESULTS	29
6	CONCLUSION AND FUTURE WORK	33
7	REFERENCES	35

PREPARATION FORMAT:

1 The dimension of the project report should be in A4 size. The project report should be bound using flexible cover of the thick white art paper. The cover should be printed in black letters and the text for printing should be identical.
2 Abstract should be one page synopsis of the project report typed double line spacing, Font Style Times New Roman and Font Size 14.
3 The impression on the typed copies should be black in colour. One and a half spacingshouldbeusedfortypingthegeneraltext.Thegeneraltextshallbetyped in the Font style 'Times New Roman' and Font size 14.
4 The listing of references should be typed 4 spaces below the heading "REFERENCES" in alphabetical order in single spacing left – justified. The reference material should be listed in the alphabetical order of the first author. Thenameoftheauthor/authorsshouldbeimmediatelyfollowedbytheyearand otherdetails.Atypicalillustrativelistgivenbelowrelatestothecitationexample quoted above.

Ariponnammal, S. and Natarajan, S. (1994) 'Transport Phonomena of Sm Sel – X Asx', Pramana – Journal of Physics, Vol.42, No.1, pp.421-425.

ABSTRACT

Fruit ripeness assessment is important to consumers, retailers, wholesalers, storage operators, and supply-chain handlers. A fruit that appears mature from its outer colour may still be unripe internally, while natural variation in cultivar, lighting, surface texture, and handling can make visual inspection inconsistent. RipenSense AI – Fruit Ripeness Scanner and Quality Assessment is an educational prototype that explores how classical Artificial Intelligence (AI) and basic computer vision can support a more structured assessment of fruit appearance.

The proposed system accepts an image through a simple application interface, processes the image using Python and OpenCV, and converts selected visual observations into facts for a rule-based expert system.Explicit IF–THEN rules are used to explain the category suggested by the system. The design discusses certainty factors as a way to express the strength assigned to a rule, while clearly distinguishing such a value from empirically measured accuracy. A Flask application layer can connect the image-processing logic, inference engine, and user-facing result.

To demonstrate broader course concepts, the project also describes classical AI components for warehouse navigation and operational planning. Breadth-first search (BFS) and depth-first search (DFS) illustrate graph traversal; A* or best-first search illustrates route selection; hill climbing with random restarts illustrates storage-slot assignment; and constraint satisfaction is modelled through backtracking, forward checking, and the minimum-remaining-values heuristic. These are educational modules associated with the fruit-handling scenario rather than evidence of a fully deployed logisticsplatform.

The preliminary prototype was checked using 16 synthetic drawings, comprising eight fruit-like examples and eight non-fruit examples. Expected outputs were observed for those drawings, but the sample is too small and artificial to establish real-world performance. An orange-coloured toy ball may be mistaken for an orange, and external colour and shape alone cannot reveal internal maturity, smell, firmness, or hidden defects. The report therefore documents the system concept, algorithms, implementation plan, test observations, constraints, and a practical future evaluation strategy. RipenSenseAIisintendedasanexplainabledecision-supportdemonstration, not a substitute for expert inspection or laboratory testing.

Keywords: Artificial Intelligence, OpenCV, Python, Flask, expert system, rule-based reasoning, certainty factor, search algorithms, constraint satisfaction, fruit ripeness.

1. INTRODUCTION

Background and Motivation

Fruit is a perishable agricultural product whose market value and usability depend partly on maturity, freshness, and handling. Consumers often judge ripeness through appearance, especially colour and visible shape. This is convenient, but it can produce inconsistent decisions. Different varieties of the same fruit may have different colours at maturity,and lighting conditions can change how the same surface appears in a photograph. Some fruit is harvested before full ripeness and continues to mature after harvest, which further complicates a single visual judgement.

In shops and warehouses, quick sorting decisions may need to be repeated across many items. A simple digital tool could make the decision process more consistent by recording observations, applying the same stated rules, and explaining why a category was suggested. Such a tool should not claim to know more than its input supports. An image provides evidence about visible appearance, not direct measurement of sugar content, firmness, aroma, internal browning, or microbial safety.

RipenSense AI is framed as a classical-AI learning project. Rather than relying on a trained deep-learning model, it represents domain assumptions as facts and explicit rules. This makes the reasoning pathway inspectable by a student, evaluator, or user. OpenCV provides basic image handling and feature extraction, while Python supports rule evaluation and algorithm demonstrations. Flask can expose the processing functions through a browser-accessible application.

Problem Statement

The central problem is that visible appearance does not always correspond to actual ripeness. A yellow fruit may be unripe, a green fruit may be naturally matureforitsvariety,andanon-fruitobjectwithasimilarcolourandoutline may be incorrectly treated as a fruit. Manual inspection also depends on the observer's experience and the environment in which the inspection takesplace.

The project addresses a narrower and testable question: can an educational application accept a fruit image, derive a small set of visible cues, apply a transparent set of rules, and present a cautious, explainable estimate? The prototype is not designed to guarantee edible quality, detect all defects, or certify a fruit as safe. Its output must communicate uncertainty and limitations instead of presenting a categorical statement as ground truth.

An additional academic problem is how to demonstrate multiple classical AI techniques within one coherent application scenario. RipenSense AI uses fruit inspection as its primary user-facing task and introduces search and constraint-satisfaction examples in associated storage and deliveryoperations.

Aim and Objectives

Aim: To design and document an explainable, rule-based AI prototype that uses basic image processing to estimate visible fruit category or ripenesscues and demonstrates selected classical AI algorithms in a fruit-handlingscenario.

- Accept an image through a straight forward application workflow.
- Validate the filetype and handle missing or unreadable image inputs.
- Use OpenCV to prepare animage andderive simple visual observations.
- Represent observation sasfacts and apply explicitIF–THEN rules.

- Explain the inferred outcome and show appropriate uncertainty warnings.
- Illustrate forward chaining,backward chaining,and logical resolution as reasoning concepts.
- Demonstrate BFSandDFS for simple warehouse-map traversal.
- Describe A*orbest-firstsearchfor routeplanning and hillclimbing for storage assignment.
- Model storage allocation as a constraint-satisfaction problem using backtracking, forward checking, and MRV.
- Record preliminary tests without over stating the significance

Scope and Significance

The scope of the prototype is deliberately limited to externally visible image cues and a small rule base. It may classify a provided image according to a configured set of observations, but it cannot directly measure internal fruit properties. The result should be interpreted as a suggestion for further inspection rather than a final quality decision.

The educational significance lies in making the reasoning visible. A machine-learning classifier may provide a label without exposing a simple human-readable rule path. A rule-based system can instead show which facts wereobservedandwhichruleproducedtheresult.Thisisusefulforteaching knowledge representation, inference, search, constraints, and the importance of evaluating systems against suitable data.

The project can be extended into a more realistic decision-support system ifa diverse dataset of real fruit images and reliable labels becomes available. Such an extension would need documented data collection, consistent labelling, independent testing, evaluation metrics, and user guidance. Any use in commercial grading would also require domain-expert validation anda much stronger measurement basis than the current synthetic test.

2. LITERATUREREVIEW

Traditional and Manual Fruit Assessment

Manualfruitassessmenttypicallycombinesseveralsensorycues.Depending on the fruit, an observer may examine colour, size, shape, surface marks, aroma, firmness, and the condition of the stem or skin. These cues are interpreted in context: the expected colour of a ripe banana differs from that of a ripe avocado, and even within one type of fruit, varieties may maturedifferently.

The strengths of manual inspection are its flexibility and the ability of an experienced person to consider context that a basic image-processing program does not represent. Its weaknesses include variation between observers, fatigue during repetitive work, and differences in lighting or viewing conditions. An image-only system cannot reproduce all human sensory information, so it should not be described as a complete replacement for manual assessment.

RipenSense AI treats manual assessment as the motivation for a structured aid. The aim is not to remove human judgement but to make selected visual observations repeatable and explainable. A responsible user interface should invite confirmation and indicate when the available image is insufficient.

Computer Vision and Image Processing

Computer vision converts image data into representations that software can process. A typical image-processing pipeline includes input validation, decoding, resizing, colour-space conversion, region selection, feature calculation, and interpretation. OpenCV provides widely used operations for imageloading,colourtransformations,thresholding,contours,andgeometricmeasurements.

Colour is a potentially useful but imperfect cue. RGB values depend on the camera,whitebalance,illumination,shadows,andsurroundingobjects.HSV orrelatedcolourspacescansometimesmakecolourrangeseasiertoexpress, but changing colour space does not eliminate the effect of lighting. A robust system would need calibration or training examples across real acquisitionconditions.

Shape cues can help identify an approximate outline or distinguish some obviously unrelated objects. However, fruit shapes overlap with toys, balls, packaging,andotherobjects.Acontourorcircularitymeasurecandescribea shape, but it cannot establish the object's identity on its own. Surface blemishes and texture may add information, yet those features also require careful design and validation.

The current project uses basic visual observations as inputs to a transparent rule base. It does not claim that these features provide a reliable universal ripeness detector. The purpose is to demonstrate the connection between image processing and symbolic reasoning while documenting the limitations that arise when simple cues are used in a complex real-world task.

Expert Systems and Classical AI

Expert system represents knowledge about a domain in a form that can be applied to a case. A simple rule has the structure IF condition THEN conclusion. The working memory stores facts about the current case, while the inference engine searches for rules whose conditions match those facts. Separating facts from rules allows the system to explain its conclusions and makes the knowledge base easier to revise.

Forward chaining begins with known facts and applies rules to derive further facts until no applicable rule remains or a goal is reached. Backwardchainingbeginswithagoalandsearchesforrulesthatcouldestablishit,then checks whether their conditions are supported. Resolution is a rule of inference used in formal logic, commonly by converting expressions into a suitable clause form and deriving a contradiction when testing whether a conclusion follows.

Certainty factors can be used in a prototype to communicate the relative strength assigned to a rule or combination of evidence. Unless those values are calibrated against a representative labelled dataset, they are not measured probabilities and must not be presented as accuracy. This distinction is particularly important for user trust.

Search algorithms solve problems by exploring possible states or paths. BFS explores nodes level by level and is complete for finite branching graphs; DFS explores a branch deeply before backtracking. A* combines path cost with a heuristic estimate of remaining cost. Hill climbing iteratively improves a candidate solution according to an objective function, but may become trapped in local optima; random restarts can reduce this risk. Constraint-satisfaction methods represent variables, domains, and constraints, then search for an assignment that satisfies them.

3. PROPOSED SYSTEM

System Overview

The proposed system is organised as a sequence of loosely coupled components. A user supplies an image; the application validates and decodes it; image-processing functions derive candidate observations; the rule engine evaluates facts; and the result page presents an estimated category, a short explanation, and a caution about uncertainty. Keeping these responsibilities separate makes the system easier to test and maintain.

Python is used for the core logic because it supports concise data handling and integration with OpenCV. OpenCV is responsible for image operations, not for making a complete semantic judgement by itself. Flask can receive the upload, call processing functions, and return a rendered result. The rule engine should be implemented as a distinct module so that rules can be reviewed independently of the web interface.

The system may also include demonstration modules for warehouse navigation and storage assignment. These modules show how classical AI can support related operational tasks. They should be presented as separate demonstrations unless they are fully integrated and tested with shared application data.

Functional Requirements

- The system shall accept a supportedimage file.
- The system shall reject empty,invalid,or unsupported uploads with a clear message.
- The system shall prepare an image for feature extraction.
- The system shall represent extracted observations as named facts.
- The system shall apply a rule base to inferanestimated result.
- The system shall display the result and a human-readable reason.

- The system shall avoid representing rule certainty as validated accuracy.
- The system shall record test outcomes for repeatable evaluation.

Non-Functional Requirements

Usability: the main workflow should require only a few steps and use plain language. A user should understand that the result is an estimate and not a safety certificate.

Maintainability: image processing, rules, web routes, and search demonstrations should be stored in separate functions or modules. Rule changes should not require rewriting the upload interface.

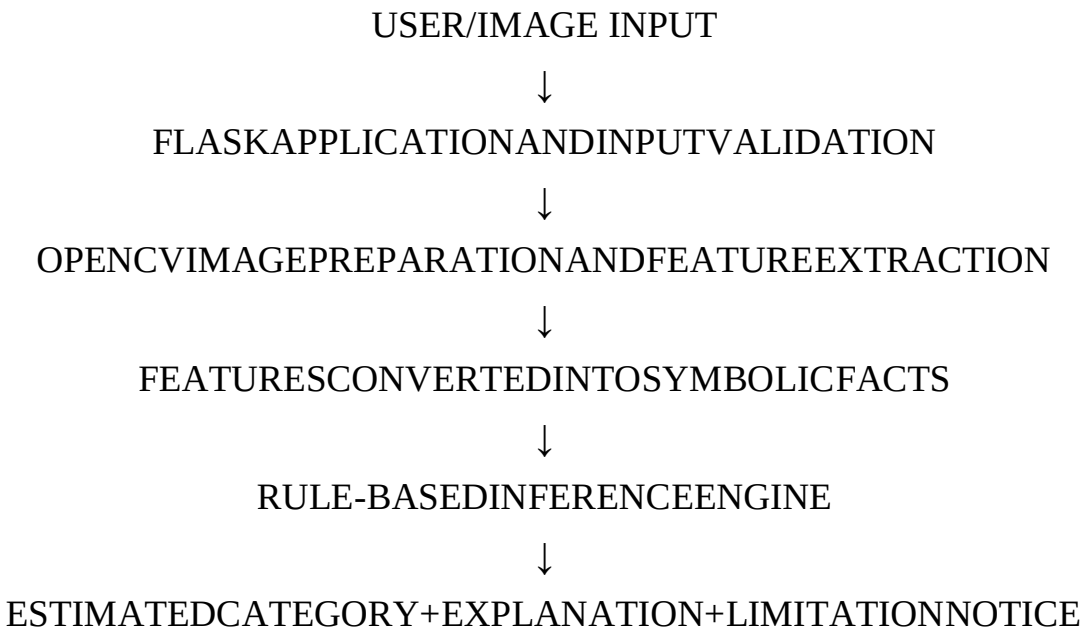
Robustness: missing files, invalid image bytes, very large files, and processing exceptions should be handled gracefully. The application should avoid crashing or returning misleading output after an input error.

Reproducibility: test images, expected observations, rule versions, and test outcomes should be recorded. A result should be traceable to the input and rules used during the test.

Privacy: images should be processed only for the purpose described to the user. If uploads are stored, the system should explain why, limit retention, and provide a deletion policy. A prototype can avoid persistent storage by processing an image temporarily and removing it after the response.

Performance: for small images and a small rule base, processing should be practical on a standard student computer. Performance should neverthelessbe measured rather than assumed, especially if image resizing, logging, or multiple users are added.

Architecture Diagram



Supporting educational modules: BFS/DFS warehouse traversal; A* or best-first delivery-route planning; hill climbing with random restarts for storage-slot assignment; and CSP search using backtracking, forward checking, and MRV.

System Workflow

1. The user opens the application and selects an image file.
2. The server verifies that a file was provided and that it can be decoded as an image.
3. Image-processing functions calculate the observations supported by the current prototype, such as broad colour ranges or simple shape descriptors.
4. The observations are translated into named facts;unavailable observations remain unknown rather than being guessed.
5. The inference engine compares the fact set against therule base.
6. The engine records the rules that fired and derives suggested category.
7. The user can review the suggestion and inspect the fruit directly;the system does not certify internal ripeness.

Knowledge Representation

Knowledge representation determines how a problem is expressed so that an inference procedure can use it. In RipenSense AI, image-processing outputs are represented as symbolic facts such as “dominant colour range detected”, “outline is approximately round”, or “image quality is inadequate”. These example fact names describe the kind of information that could be represented; they do not imply that the current prototype has a comprehensive fruit-recognition model.

Rules connect observations to cautious conclusions. For example, a rule may state that if the image is valid, the detected shape is compatible with a configured fruit profile,and colour observations meet a selected range,then

the system may suggest a candidate fruit category for further inspection. A separate rule should detect conflicting cues or low-quality input and reduce the strength of the conclusion.

Facts should distinguish TRUE, FALSE, and UNKNOWN wherever possible. The absence of evidence for a feature is not necessarily evidence that the feature is absent. This matters when an image is blurred, poorly lit,or partially occluded. A safe rule base should have a fallback outcome such as “unable to assess from this image” rather than forcing every input into a ripe or unripe label.

Illustrative Rule Base

Rule R1 — Input validity: IF the uploaded file cannot be decoded as an image, THEN return an input error and do not run inference.

Rule R2 — Insufficient evidence: IF image quality is inadequate or essential observations are unknown, THEN return “insufficient visual evidence”.

Rule R3 — Candidate fruit appearance: IF a valid image contains a configured fruit-like outline and compatible colour observations, THEN suggest a candidate fruit appearance class, subject to uncertainty.

Rule R4 — Conflicting cues: IF one observation supports a category but another contradicts it, THEN mark the result as uncertain and show the conflicting cues.

Rule R5 — Ripeness cue: IF the selected fruit profile has a documented colour rule and the observed range matches it, THEN report that the visible colour cue is consistent with that profile; do not infer internal ripeness.

Rule R6 — Human review: IF the result is uncertain or the fruit profile is unsupported, THEN ask the user to verify the fruit and avoid a definitive ripeness label.

These are illustrative rules for explaining the proposed architecture. Actual thresholds and fruit-specific rules must be specified, tested, and documented before a deployed implementation can be evaluated.

Certainty Factors

A certainty factor is a value associated with a piece of evidence or a rule outcome to communicate relative support within a reasoning system. It can help the interface distinguish a strong rule match from a weak or conflicting match. However, the value is meaningful only within the assumptions usedto define it.

In this project, certainty factors should be labelled as rule-based estimates. They must not be called probability of ripeness, model accuracy, or confidence calibrated to real-world outcomes unless a separate calibration study supports that interpretation. A responsible interface can show the evidence behind a result and explain that the number reflects configuredrules rather than validated statistical performance.

Methodology and Algorithms

Image Processing Sequence

8. Image decoding: read the suppliedfile and verify that a valid image matrix is produced. Reject unsupported or corrupt data.
9. Resizing :scale very large images to a controlled working sizewhile preserving the aspect ratio. This can reduce processing time and standardise operations.
10. Colour analysis:convert the image into a selected colourspace and calculate simple statistics or colour-range proportions. The chosen thresholds should be documented and treated as provisional.
11. Shape analysis: where suitable, use a mask or contour to estimate broad shape descriptors.Shape descriptors are supporting cues only andcannot establish fruit identity by themselves.
12. Fact generation:convert measured observations into named factswith explicit values or ranges. Store unknown or unreliable measurements separately.
13. Inference:pass the facts to the rule engine and collect both the conclusion and the list of rules that contributed to it.

Forward and Backward Chaining

Forward chaining starts with available observations and repeatedly applies any rule whose conditions are satisfied. It is useful when the system needs to derive all supported conclusions from the current image facts. Backward chaining begins with a question, such as whether a particular candidate class is supported, and checks the rules that could establish that goal. It may avoid evaluating unrelated rules. The prototype can use one method operationally and demonstrate the other with a small trace for educational purposes.

Search Algorithms

Breadth-first search (BFS): BFS explores the nearest unvisited nodes first, using a queue. In a warehouse grid where each movement has equal cost, BFS can find a path with the fewest steps if a path exists and the graph is finite. It can consume considerable memory because it stores the frontier.

Depth-first search (DFS): DFS follows one branch as far as possible before backtracking, typically using a stack or recursion. It is useful for exploring connectivity but does not generally guarantee the shortest path. Its memory use can be lower than BFS in some graph structures, although its behaviour depends on the graph and implementation.

A* search: A* evaluates a node using f(n) = g(n) + h(n), where g(n) is the cost already incurred and h(n) estimates the remaining cost. With an appropriate admissible heuristic and suitable conditions, A* can find an optimal path. For a grid with four-directional movement and uniform costs, Manhattan distance is a common heuristic.

Best-first search: best-first search selects a frontier node according to an evaluation function,often a heuristic.It can quickly find a plausible route but is not automatically optimal. The report distinguishes it from A* because the evaluation functions and guarantees differ.

Hill Climbing and Random Restarts

Hill climbing begins with a candidate storage-slot assignment and evaluates neighbouring assignments. A neighbour might be produced by moving afruit batch to a different compatible slot. The algorithm accepts a neighbour when it improves an objective score, such as reducing travel distance while respecting capacity and compatibility requirements.

Because hill climbing uses local improvement, it may stop at a local optimum.Randomrestartsrunthesearchfrommultiplestartingassignments

and retain the best valid solution found. The quality of the result depends on the objective function, neighbourhood definition, constraints, and number of restarts. It does not guarantee a globally optimal solution in general.

Constraint Satisfaction Problem

A constraint-satisfaction problem (CSP) consists of variables, domains, and constraints.Inastorage-allocationexample,variablesrepresentfruitbatches, domains represent available slots, and constraints express capacity, separation of incompatible items, temperature requirements, or other handling rules. A solution is an assignment that satisfies all specified constraints.

Backtracking assigns a value to a variable and checks whether the partial assignment remains consistent. If a constraint is violated, the algorithm undoes the most recent choice and tries another value. Forward checking removes domain values that are no longer feasible for unassigned variables after each assignment. This can detect failure earlier than waiting until a complete assignment is formed.

The minimum-remaining-values (MRV) heuristic selects the unassigned variable with the fewest legal values remaining. It attempts to expose a contradiction early by tackling the most constrained variable first. MRV changes the order of search but does not change the underlying validity of asolution.

For a practical storage system, constraints must come from verified operational requirements. The educational model can demonstrate algorithm behaviour, but it should not invent food-safety rules or assume that all fruit varieties share identical storage needs.

Assumptions and DesignDecisions

- The current visual decision is based on external appearance only.
- Fruit-specific thresholds require documentation and validation.
- Unknown or conflicting evidence should lead to a cautious output.
- Theapplicationshouldkeeptheruleengineseparatefrompresentationlogic.
- Search andCSP modules are educational demonstrations unless integrated with live inventory data.
- Syntheticimagessupportfunctionalcheckingbutcannotrepresentreal-world variability.

4. MODULES DESCRIPTION

Image Input and Validation Module

The image input module is the entry point for the main application. It provides a file-selection control and gives the user a clear indication of whether the image has been received. A reliable upload process is important because later stages depend on correctly decoded pixel data. A corrupt file should never be silently treated as an empty or black image.

Validation includes checking that a file was selected, verifying the extension or detected media type, limiting file size, and attempting to decode the contents. File extensions alone are not sufficient evidence that a file is avalid image, so the application should verify the decoded result. The system should also handle unexpected image dimensions and report a readable error rather than exposing internal exception details.

The module should avoid unnecessary retention of uploaded images. If the application temporarily writes an upload to disk, it should use a safe temporary location and remove the file after processing. Persistent image storage should only be introduced if there is a clear need and an appropriate user-facing notice.

Expected output from this module is either a valid image object ready for processing or a controlled error message. The module does not decide whether the image contains fruit and does not infer ripeness.

Image Processing and Feature Extraction Module

The image-processing module converts the raw image into a small collection of observations. Depending on the implemented version, this may include resizing, colour-space conversion, broad colour statistics, segmentation by configured thresholds, and basic contour measurements. The module should return structured data rather than a final natural-language answer, so the rule engine can operate independently.

Colour measurements can be affected by warm or cool lighting, shadows, camera exposure, reflections, and background objects. A crop or segmentation step can reduce some background influence, but it may alsofail when the fruit blends into the background. For this reason, the application should avoid interpreting a single colour measurement as definitive evidence.

Shape features can describe whether a visible object is roughly circular, elongated, or irregular. Many unrelated objects share those characteristics. The shape module should therefore produce descriptive observations, not a guaranteedobjectidentity.Whentheimagecontainsmultipleobjects,abasic implementation may also fail to determine which object the user intended to scan.

The output format can include a dictionary of named features, quality flags, and processing notes. A quality flag can indicate low resolution or an uncertain segmentation. These flags should influence the inference process and may trigger an “unable to assess” response.

Rule-Based Expert System Module

The rule-based expert system is the main classical-AI reasoning component. It receives facts from image processing, compares them against rule conditions, and derives one or more candidate conclusions. The knowledge base should be kept in a readable format so that an evaluator can inspect which assumptions govern a result.

A simple rule engine may evaluate rules sequentially. For each rule, itchecks whether the required facts exist and whether their values satisfy the conditions. If a rule fires, its conclusion is added to working memory and the rule identifier is recorded. The process can continue until no new facts are derived or a predefined stopping condition is reached.

An explanation facility should link the final result to the observations that supported it. For example, the result page may say that a candidate class was suggested because the image passed validation and certain visible cues matched a configured profile. It should also show any contradictory cues or missing information. This explanation is more useful than presenting a labelalone.

Rule conflicts need explicit handling. If two rules support different outcomes,the system should use a documented priority or report uncertainty. It should not arbitrarily choose a confident label without disclosing the conflict. A fallback category is important because the knowledge base will not cover every fruit variety, image condition, or non-fruit object.

Search and Constraint-Satisfaction Modules

The warehouse-navigation demonstration represents locations as nodes and permitted movements as edges. BFS and DFS traverse the same graph with different exploration orders. A test can compare whether each algorithm finds a path and how many nodes it visits. For equal-cost edges, BFS is appropriate when the shortest number of steps is required; DFS is useful for demonstrating exploration and backtracking.

The route-planning demonstration adds path costs and a heuristic. A* combines accumulated cost with an estimate of remaining distance. Its behaviour depends on the graph, cost model, and heuristic. The interface should show the route on a simple grid and label the selected algorithm sothe result can be compared and explained.

The storage-slot assignment demonstration uses hill climbing to improve a candidate arrangement. A score might penalise travel distance, capacity overflow, or incompatible assignments. Hard constraints should be checked separately so that an apparently high-scoring but invalid arrangement is not accepted. Random restarts help explore different initial arrangements.

The CSP module models each batch as a variable and each compatible slotas a domain value. Backtracking, forward checking, and MRV can be compared on small scenarios. These examples are intended to teach algorithmic reasoning. They do not substitute for verified warehouse procedures or food-storage standards.

UserInterface,Results,and Explanation Module

The user interface should be minimal and task-focused. The user selects an image, starts the scan, and receives the estimated outcome. The page should distinguish an input error from a successful analysis that found insufficient evidence. These are different situations: an invalid upload cannot be analysed, while an uncertain image may have been processed successfully but does not support a reliable conclusion.

The results view should contain the estimated category, a short explanation of the rules that fired, key visible observations, and a limitation notice. If a certainty factor is shown, the page should define it as a rule-based indicator and avoid describing it as validated probability or accuracy. If no supported conclusion can be derived, the page should state that directly.

Accessibility and readability matter for a student project. Labels should be clear, text should have sufficient contrast, and the result should not rely on colour alone to communicate status. A textual description such as “uncertain result” is more accessible than a coloured badge with no explanation.

For repeatable evaluation, the system can log a test identifier, rule version, input type, derived facts, output category, and expected output where applicable. Logs should avoid collecting personal information that is unnecessary for evaluation. If real user images are used in future testing, consent and retention expectations should be clearly described.

5. IMPLEMENTATION AND RESULTS

Experimental Setup

The proposed application stack consists of Python for core logic, OpenCV for image processing, and Flask for the web application layer. Python functionscanbeorganisedintoseparatemodulesforinputvalidation,feature extraction, rule evaluation, explanation generation, and search demonstrations. The separation allows each component to be tested without requiring the entire web application to run.

The test environment should record the Python version, package versions, operatingsystem,imagedimensions,andruleconfiguration.Thisreportdoes not claim a fully benchmarked production environment; these items arelisted as reproducibility details to record when the prototype is executed and evaluated systematically.

The preliminary test described for this project used 16 synthetic drawings: eight fruit-like drawings and eight non-fruit drawings. The purpose was to check basic flow and observe whether the prototype returned the expected categories for those specific inputs. The sample is intentionally described as synthetic because drawings do not capture the range of real fruit appearance, camera quality, lighting, occlusion, variety, or maturity states.

No claim of real-world classification accuracy can be derived from these 16 drawings. A larger and independently labelled set of real images would be required. It would also be necessary to define the ground truth carefully: “ripe” can mean different things for different fruit types, and external appearance may not be sufficient to determine the correct label

Test Casesand Procedure

A useful test plan should cover normal inputs, invalid inputs, ambiguous inputs, and cases that challenge the assumptions of the rule base. The table below describes a recommended functional test set. It is a test plan, not a claim that every listed scenario has already been executed.

TestID	Scenario	Expected behaviour
T01	Valid fruit-likesynthetic drawing	Process the imageand returnthe configured candidate result.
T02	Valid non-fruitdrawing	Avoid a fruit conclusion if the configured facts do not support it.
T03	Missing upload	Show a clear input error and stop processing.
T04	Corrupt image file	Reject the filewithout crashing.
T05	Very small or blurred image	Flag insufficient evidence wheres upported.
T06	Orange-coloured toy ball	Demonstrate possible false positive;do not claim robust recognition.
T07	Unusual fruit colour or variety	Returnuncertaintyifnorule profile applies.
T08	Conflicting feature observations	Explaintheconflictorreturnan uncertain outcome.
T09	Ware house path exists	BFS should find a path in afinite unweighted graph.
T10	Warehouse pathblocked	Search should report no route if no valid path exists.
T11	Storage slot capacityexceeded	Assignment should be rejected by hard constraints.
T12	CSP withconstrained variable	MRVshouldselectthevariable with fewest legal values.

Preliminary Results

For the 16 synthetic drawings used in the preliminary functional check, the expected outputs were observed. This observation supports only the limited statement that the prototype pipeline behaved as expected on thoseexamples. It does not establish a reliable estimate of performance on photographs of real fruit.

Synthetic drawings often have clean outlines, uniform colour, and limited background clutter. Real images contain reflections, shadows, colour casts, damaged surfaces, leaves, labels, packaging, and multiple objects. These differences can cause the same rule to behave differently. The synthetic set also does not provide a sufficient number of examples for meaningful per-class precision, recall, or a statistically useful accuracy estimate.

A known weakness is the possibility of a false positive when a non-fruit object resembles a fruit in colour and shape. An orange-coloured toy ball is anillustrativeexample.Thisweaknessisnotanunexpectedsoftwareerror;it follows from the limited evidence used by the rules. A transparent resultpage should make this limitation clear.

The prototype is most successful as a teaching demonstration of the pipeline from image input to feature observations, symbolic facts, rule matching, and explanation. It also provides a coherent scenario for demonstrating search and CSP algorithms. Any claim beyond this scope would require additional evidence.

Metrics for FutureEvaluation

- Accuracyistheproportionofalltestedcasesclassifiedcorrectly,butit can be misleading when class distributions are unbalanced.

- Precision measures how many predicted positive cases are actually positive.Itisimportantwhenfalsepositives,suchasidentifyingatoyas fruit, are costly.
- Recall measure show many actual positive cases are detected.It helps quantify how often genuine fruit images are missed.
- F1-scorecombinesprecisionandrecallthroughtheirharmonicmean.Itis useful when both false positives and false negatives matter.
- A confusionmatrix should show counts for each actual and predicted class. For multi-class fruit categories, per-class measures should be reported rather than one aggregate value alone.
- Latency should be measured overrepeated runson documented hardware. A user study may also assess whether people understand the explanation and limitation notice.

Results Discussion and Limitations

The principal limitation is the gap between visual appearance and internal maturity. Fruit ripeness is a biological condition that may involve firmness, aroma, soluble solids, acidity, starch conversion, and internal changes. An ordinary RGB photograph cannot directly measure all these properties. A system that uses only colour and shape must therefore avoid presenting its output as a definitive maturity assessment.

Another limitation is variation between fruit types. A universal colour threshold is unlikely to work for every fruit, cultivar, growing region, or stage of development. A fruit-specific rule base could be more appropriate, but it would require reliable domain knowledge and representative images. Unsupported categories should be detected and reported as unsupported.

Lighting and capture conditions can affect colour features. A camera may apply automatic white balance, and a glossy surface may create highlights that distort measured colour. Background segmentation can fail when the background has a similar colour to the object. Future testing should intentionally include these conditions rather than only ideal images.

Rule maintenance is also important. If a rule is changed to improve one example,itmaycausearegressiononanother.Thesystemshouldmaintaina versioned test set and rerun all cases after changes. The explanation facility should identify the rule version used for a result so that test outcomes can bereproduced.

Finally, the logistics algorithms are demonstrations unless supported by real operational data. A route-planning result depends on a map and cost model;a storage assignment depends on capacity and compatibility constraints. These values should be sourced from actual requirements before practicaluse.

Ethical and Practical Considerations

- Do not describe a visual estimate as a guarantee of safety,freshness,oredibility.
- Do not represent hand-set certainty values as measured accuracy.
- Explain what image information is processed and whether uploads are retained.
- Avoid making commercial grading decisions without domain validation.
- Provide a way to report in correctresults during future testing.
- Ensure that limitations are visible alongside the output, not hidden in documentation.

6. CONCLUSION AND FUTUREWORK

Conclusion

RipenSenseAI–Fruit Ripeness Scanner and Quality Assessment presents an educational approach to combining basic computer vision with classical Artificial Intelligence. The proposed workflow receives an image, derives limited visual observations, represents them as facts, applies a transparent rule base, and returns an estimated category with an explanation. Python, OpenCV, and Flask provide a practical technology stack for implementing the prototype.

The project also connects the main application scenario to classical AI topics throughBFSandDFStraversal,A*orbest-firstrouteplanning,hillclimbing with random restarts, and CSP techniques including backtracking, forward checking, and MRV. These components help demonstrate how different AI methods address distinct kinds of problems.

The preliminary functional check used 16 synthetic drawings and observed expected outputs for those examples. This is not evidence of real-world accuracy. The toy-ball false-positive example, sensitivity to lighting and background, and inability to measure internal properties show why theoutput must remain cautious and explainable.

The main outcome is therefore a documented, interpretable prototypeconcept and an evaluation plan, rather than a production-ready fruit-quality instrument. The project reinforces that responsible AI development requires clear scope, testable requirements, transparent reasoning, honest reporting, and evaluation on data representative of the intended use.

Future Work

14. Collect a labelled dataset of real fruit photographs across varieties, ripenessstages,cameras,lightingconditions,backgrounds,andcapture distances. Define labelling rules with help from domain experts.
15. Splitthedatasetintotrainingordevelopmentdataandanindependenttest set. Avoid placing near-duplicate images from the same fruit or capture session in both sets, as this can overstate generalisation.
16. Compare the rule-based baseline with more advancedcomputer-vision methods only when there is a suitable dataset and a fair evaluation procedure.Keep a simple baseline for interpretability and comparison.
17. Explore additional non-destructive measurements, such as firmness instrumentsorspectralsensing,iftheprojectobjectiveexpandsfrom visible appearance to internal maturity.
18. Improve image quality checks, background handling, fruit-region selection,and support for multiple objects.Add an explicit“unknown/ unsupported fruit” result.
19. Calibrate any certainty output against empirical data before calling it a probability.Publish the definition,calibration method,and limitations of the value.
20. Build a repeatable automated test suite for file validation, feature extraction,rules,routesearch,andCSP assignments.Record metrics and failures across software revisions.
21. Add accessible interface design, clear privacy notices, safe upload handling,and an optional feedback workflow for users to flag in correct suggestions.
22. For the warehouse demonstrations, use a documented grid, route costs, storage capacity data, and operational constraints. Compare search algorithms using pathcost,nodes expanded, runtime,and memory where feasible.

7. REFERENCES

1. Bratko, I. (2011). *Prolog Programming for Artificial Intelligence*. 4th ed. Pearson.
2. Flask Documentation. (n.d.). *Flask: A Python Microframework*. Flask Documentation
3. Gonzalez, R. C. and Woods, R. E. (2018). *Digital Image Processing*. 4th ed. Pearson.
4. Nilsson, N. J. (1998). *Artificial Intelligence: A New Synthesis*. Morgan Kaufmann.
5. OpenCV Documentation. (n.d.). *OpenCV: Open Source Computer Vision Library*. OpenCV Documentation
6. Poole, D. and Mackworth, A. (2017). *Artificial Intelligence: Foundations of Computational Agents*. 2nd ed. Cambridge University Press.
7. Python Software Foundation. (n.d.). *Python Documentation*. Python Documentation
8. Rich, E. and Knight, K. (1991). *Artificial Intelligence*. 2nd ed. McGraw-Hill.
9. Russell, S. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson.
10. Winston, P. H. (1992). *Artificial Intelligence*. 3rd ed. Addison-Wesley.